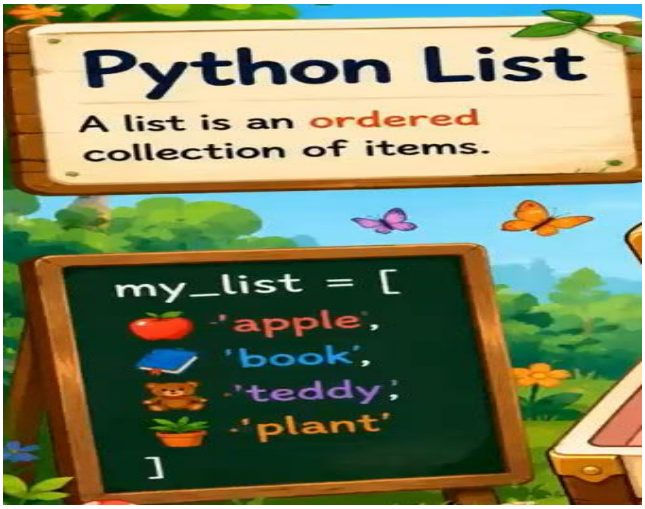
بسمه تعالی

کار با لیست ها و اساس کار با آنها:

همانطور که در شکل زیر مشاهده می کنید در داخل صندوقچه یک لیست پایتونی که حاوی ٤ عنصر یا آیتم مختلف می باشد قرار داده شده است،وقتی که به صندوقچه با دقت نگاه می کنیم می بینیم که هر آیتم دارای یک شماره می باشد،مثلا آیتم سیب دارای اندیس یا شماره یا اینکس صفر می باشد و یا کتاب دارای اندیس یا ایندکس ۱ می باشد و به همین ترتیب،حال اگر به تابلوی سمت راستِ کنار صندوقچه نگاه کنیم در داخل این تابلو نوشته است که هر آیتم درون این لیست دارای یک شماره ی index می باشد که این شماره ها از عدد صفر شروع می شود.



نکته: همانطور که در شکل فوق مشاهده می کنید می بینیم که نوعِ آیتم های درون این صندوقچه با یکدیگر متفاوت هستند و از یک جنس نیستند،پس نتیجه می گیریم که درون لیست ها می توان داده های با انواع گوناگون را ذخیره نماییم.



نکته:اگر به تابلوی سمت چپ صندوقچه نگاه کنیم،می بینیم که نحوه یا فرمت تعریف کردن یک لیست را در زبان برنامه نویسی پایتون به ما نشان می دهد.یعنی ابتدا نام متغیر دلخواهی را تعریف کرده به عنوان مثال (my_list) سپس علامت مساوی را قرار داده و بعد از مساوی کروشه ی باز و بسته و در داخل کروشه عناصر خود را در درون علامت کتیشن قرار داده ایم و مابین عناصر نیز از علامت ویرگول استفاده کرده ایم،از طرفی همانطور که به ترتیب، عناصر را در داخل کروشه قرار داده ایم به همان ترتیب هم آن عناصردر داخل صندوقچه قرار گرفته اند بنابراین در تابلوی سمتِ چپ و بالاییِ شکل، به ما می گوید که یک لیست مجموعه ای مرتب از اقلام است بنابراین گفته می شود که لیست ها ordered ،یعنی مرتب و منظم هستند.

در زیر خلاصه ای از توضیحاتی که در مورد لیست ها گفته ایم را بیان می کنیم.

ویژگی لیست‌ها

Ordered: لیست‌ها دارای ترتیب مشخص هستند: عناصر لیست به همان ترتیبی که وارد می‌شوند ذخیره می‌گردند و این ترتیب تغییر نمی‌کند، مگر برنامه‌نویس آن را تغییر دهد.

Mixed data: لیست انواع مختلف داده را می‌تواند نگهداری کند.

Indexed: عناصر لیست دارای ایندکس هستند؛ یعنی هر عنصر یک شماره دارد و اولین عنصر لیست دارای اندیس صفر است.

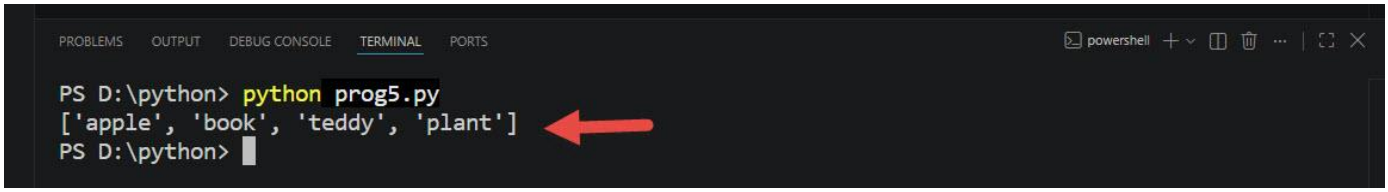
Mutable: لیست قابلیت تغییر دارد؛ یعنی بعد از ایجاد لیست، می‌توان عناصر آن را ویرایش، حذف یا اضافه کرد.

از بین ویژگی های فوق همانطور که در ویژگی Mutable توضیح داده شده در لیست های پایتونی بعد از اینکه لیستی را ایجاد می کنیم می توانیم عناصر آن را ویرایش،حذف و یا اضافه کنیم در واقع می توان گفت این ویژگی یکی از ویژگی های خیلی مهمِ لی ست ها در زبان پایتون می با شد، در زبان های برنامه نوی سی دیگر مانند زبان های C ، C++ و جاوا وقتی که شما یک آرایه(آرایه یک چیزی مشابه با همین لیست می باشد که در زبانهای دیگر به آن آرایه گفته می شود) تعریف می کنید دیگر امکان اینکه شما بتوانید سایز آرایه را تغییر بدهید را ندارید و یا امکان اینکه یک عنصر را از داخل آرایه حذف کنید را ندارید و یا امکان اینکه عنصر جدیدی را در وسط ان آرایه درج کنیم را ندارید،بنابراین ویژگی Mutable یکی از مزایای لیست ها در زبان برنامه نویسی پایتون نسبت به آرایه ها در زبان های برنامه نویسی دیگر محسوب می شود.

ذکر یک مثال از تعریف لیست در محیط برنامه نویسی:

```
my_list = ["apple", "book", "teddy", "plant" ]
print(my_list)
```

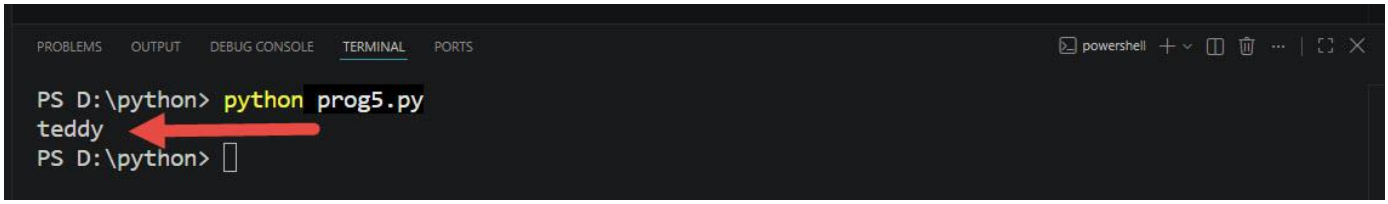
خروجی:



نکته : آیا می توانیم به تک تک عناصر لیست دسترسی داشته باشیم؟پاسخ:بله،باید شماره عنصر را در داخل کروشه بنویسیم.

```
my_list = ["apple", "book", "teddy", "plant" ]
print(my_list[2])
```

خروجی:



اگر در مثال بالا print(my_list[3])) بنوییسیم خروجی برنامه plant خواهد شد.

نکته:اگر شماره ای خارج از محدوده در دستورت print وارد کنیم مثلا اگر بنویسیم:

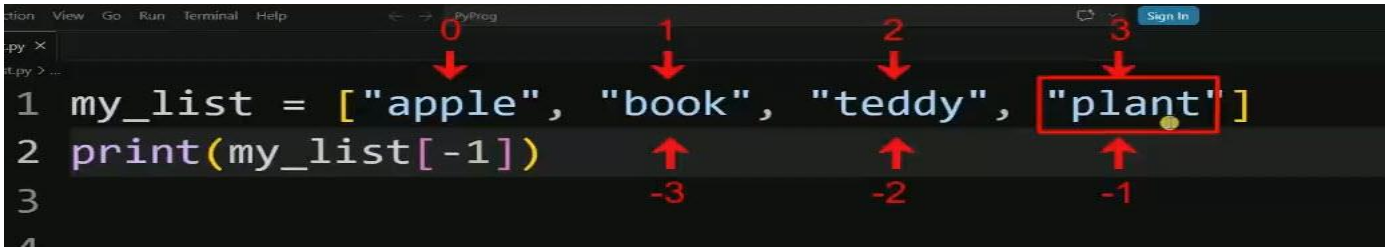
```
my_list = ["apple", "book", "teddy", "plant" ]
print(my_list[4])
```

خروجی ما با خطا مواجه خواهد شد:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                powershell

PS D:\python> python prog5.py
Traceback (most recent call last):
  File "D:\python\prog5.py", line 2, in <module>
    print(my_list[4])
          ~~~~~~~^^^
IndexError: list index out of range
PS D:\python>
```

نکته مهم :اگر در کرو شه ی جلوی نام متغیر از نوع لی ست در د ستور print عدد ۱- را قرار دهیم در ست است که خارج از محدوده ی ۰ تا ۳ است اما به ما خروجی plant را میدهد. در زبان پایتون برای دسترسی به آخرین عنصر از یک لیست کافیست که از عدد ۱- استفاده کنیم و خلاصه برای دسترسی به عناصر لیست از آخرِ لیست می توانیم از ایندکس های منفی استفاده کنیم.



خروجی:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                powershell

PS D:\python> python prog5.py
plant
PS D:\python>
```

```
my_list = ["apple", "book", "teddy", "plant" ]
print(my_list[-1])
print(my_list[3])
```

خروجی:

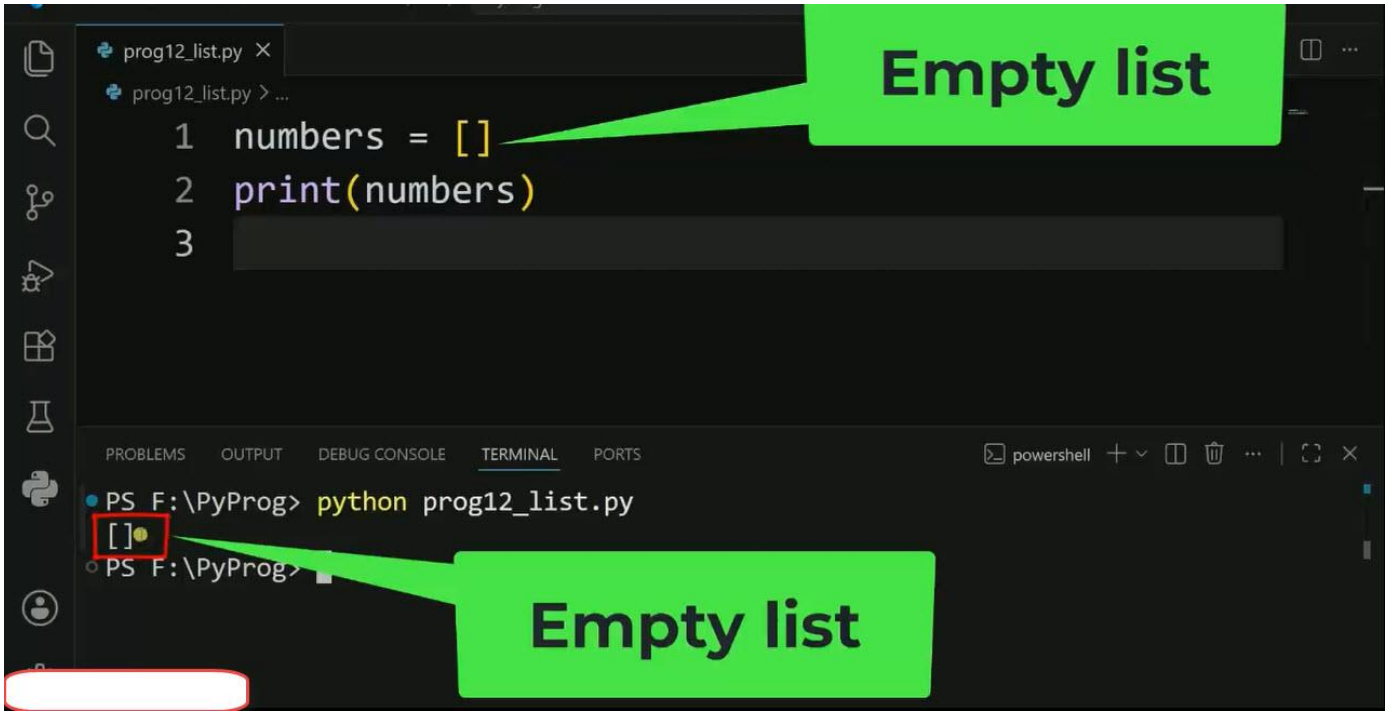
```
PS D:\python> python prog5.py
plant
plant
PS D:\python>
```

خلاصه مبحث گفته شده در رابطه با لیست ها تا اینجای کار(شکل زیر)



کار با لیست ها: در این جلسه قصد داریم در محیط برنامه نویسی با لیست ها و قابلیت های آن بیشتر آشنا شویم.

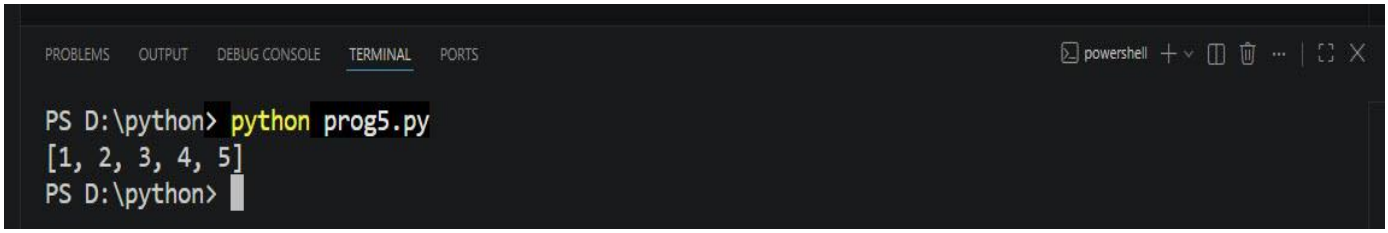
نکته:اگر متغیر از نوع لیست را تعریف کنیم و داخل لیست هیچ مقداری را قرار ندهیم و سپس آن لیست را چاپ نماییم برنامه برای ما یک لیست خالی را در خروجی نشان می دهد.(شکل زیر)



حال اگر لیست را مقدار دهی کنیم داریم:

```
numbers = [1, 2, 3, 4, 5]
print(numbers)
```

خروجی:



نکته: جلسه قبل گفتیم که عناصر لیست می توانند از هر نوعی باشند،در مثال زیر یک متغیر از نوع لیست با نام mixed_list تعریف کرده ایم(نکته جانبی:به اینگونه تعریف کردنِ نام متغیر که نام متغیر را با علامت underline تعریف می کنیم روش Snake case می گوییم.) سپس داخل متغیر mixed_list مقادیری از انواع داده قرار می دهیم و سپس آن مقادیر را چاپ می کنیم.(نوع داده ها به ترتیب رشته ای،عدد صحیح،عدد اعشاری،و نوع داده منطقی،نوع داده منطقی نوع داده ای است که همیشه یا برابر با True می باشد و یا برابر است با False و حتما باید حرف اول True یا False با حروف بزرگ نوشته شود.)

```
mixed_list = ["ali", 2, 3.14, True]
print(mixed_list)
```

خروجی:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                powershell
PS D:\python> python prog5.py
['ali', 2, 3.14, True]
PS D:\python>
```

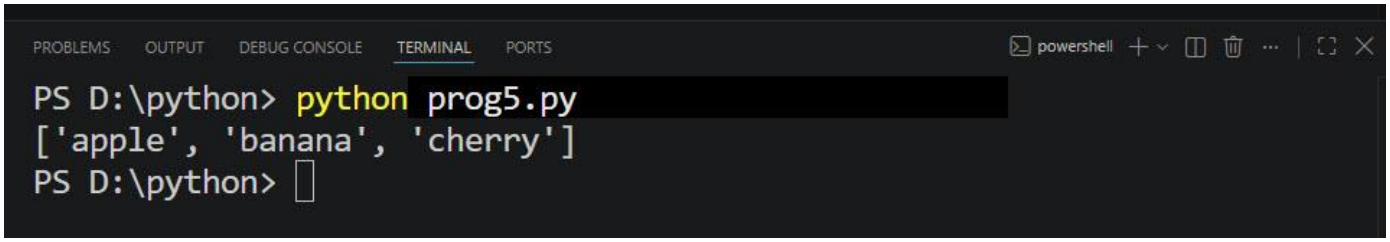
به این حالت لیست که می تواند داده های مختلف و از هر نوعی را داخل خود قرار دهد اصطلاحا Mixed Data گفته می شود.

مثال:مطابق شکل زیر یک لیست بسازید که حاوی مقادیر داخل صندوقچه باشد.



```
fruits = ["apple", "banana", "cherry"]
print(fruits)
```

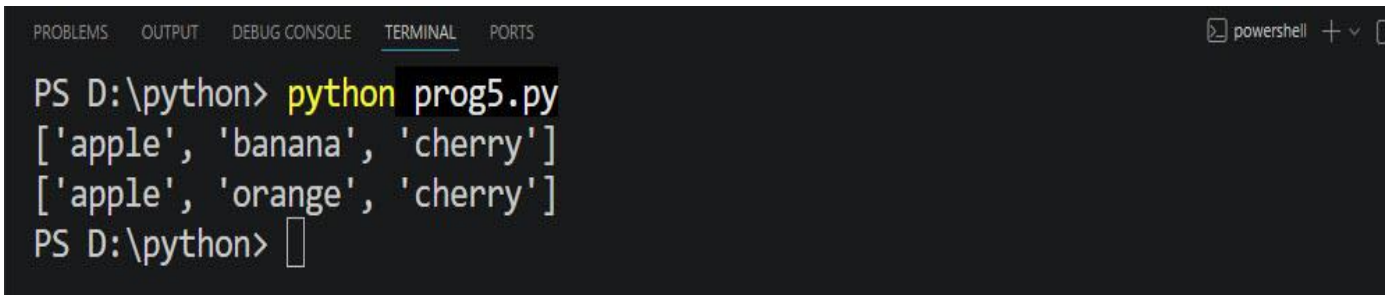
خروجی تا اینجای کار:



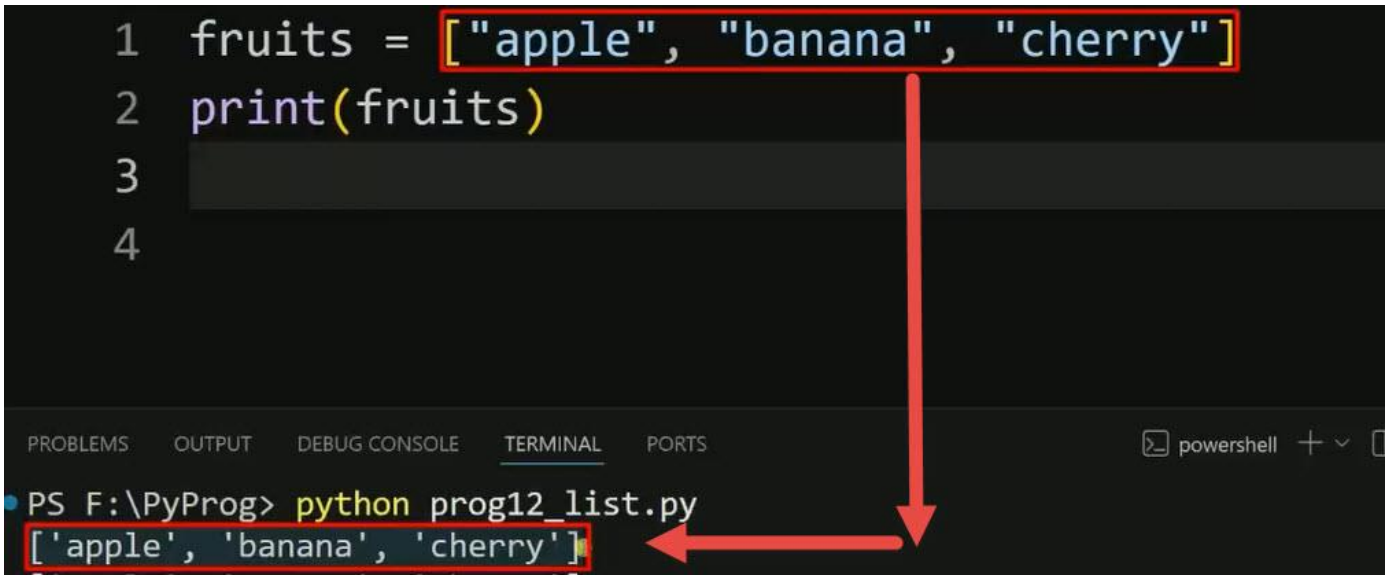
حال می خواهیم ببینیم که آیا می توانیم مقادیر عناصر داخل لیست را تغییر بدهیم؟ اگر یادتان باشد در جلسه قبل گفته بودیم که لیست هایی که در زبان پایتون تعریف می کنیم ویژگی mutable دارند یعنی می توانیم سایز لیست و همچنین مقادیر عناصر لیست را تغییر دهیم،به عنوان مثال ما قصد داریم مقدار عنصر شماره ۱ لیست را تغییر دهیم یعنی به جای مقدار banana (موز) مقدار پرتغال را جایگزین کنیم،بنابراین این کار را به صورت زیر باید انجام دهیم:

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
fruits[1] = "orange"
print(fruits)
```

حال اگر برنامه را اجرا کنیم خواهیم داشت:به خروجی print اول و print دوم دقت کنید.



نکته: در جلسه قبل اشاره کرده بودیم که عناصر لیست ordered هستند یعنی با همان ترتیبی که درون لیست مقداردهی می شوند با همان ترتیب نیز در حافظه قرار می گیرند.



با توجه نکته ای که گفتیم ما می آییم یک لیست دیگری تعریف می کنیم و در لیست دوم مکان موز و سیب را نسبت به لیست اول جابجا می کنیم و می خواهیم ببینیم که آیا با توجه به این جابه جایی لیست یک برابر با لیست دوم هست یا نه؟

```
fruits1 = ["apple", "banana", "cherry"]
fruits2 = ["banana", "apple", "cherry"]
print(fruits1 == fruits2)
```

همانطور که مشاهده می کنید هر دو لیست از نظر تعداد عناصر با هم برابر هستند و حتی تک تک مقادیر هم در هر دولیست به صورت یکسان تعریف شده اند و فقط ما آمدیم جای دو عنصر را با هم جابه جا کرده ایم در خط سوم به کمک علامت == می توانیم دو مقدار را با هم مقایسه کنیم یعنی خط سوم به ما می گوید که آیا لیست ۱ مساوی با لیست۲ هست یا نه؟اگر دو لیست برابر باشند ما باید در خروجی جواب True را ببینیم و اگر برابر نباشند باید جواب False را مشاهده کنیم،بنابراین با توجه به این توضیحات داریم:

خروجی:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\python> python prog5.py
False
PS D:\python>
```

پس با توجه به خروجی فوق متوجه شدیم که ترتیب عناصر لیست برای ما بسیار مهم می باشد و کوچکترین جابه جایی باعث تغییر در لیست خواهد شد.

نکته: اگر بخواهیم بدانیم که تعداد عناصر یک لیست چند تا می باشد برای این منظور از تابعی به نام len() در داخل دستور print استفاده می کنیم.

```
fruits = ["apple", "banana", "cherry"]
print(len(fruits))
```

خروجی:

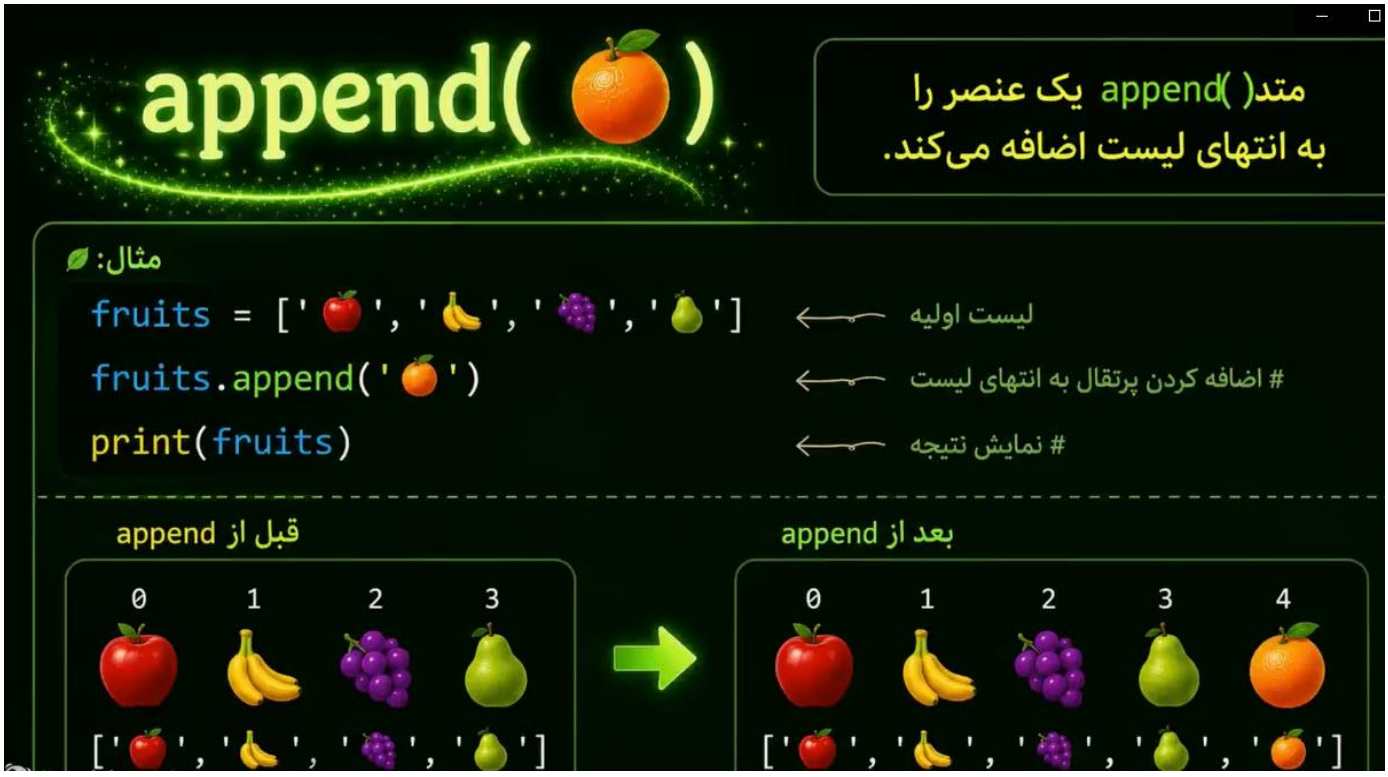
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS D:\python> python prog5.py
3
PS D:\python>
```

استفاده از متد های Append و insert جهت اضافه کردن آیتم های جدید به لیست

الف)می خواهیم مطابق شکل زیر آیتم جدیدی را به کمک متد Append به انتهای لیست اضافه کنیم.



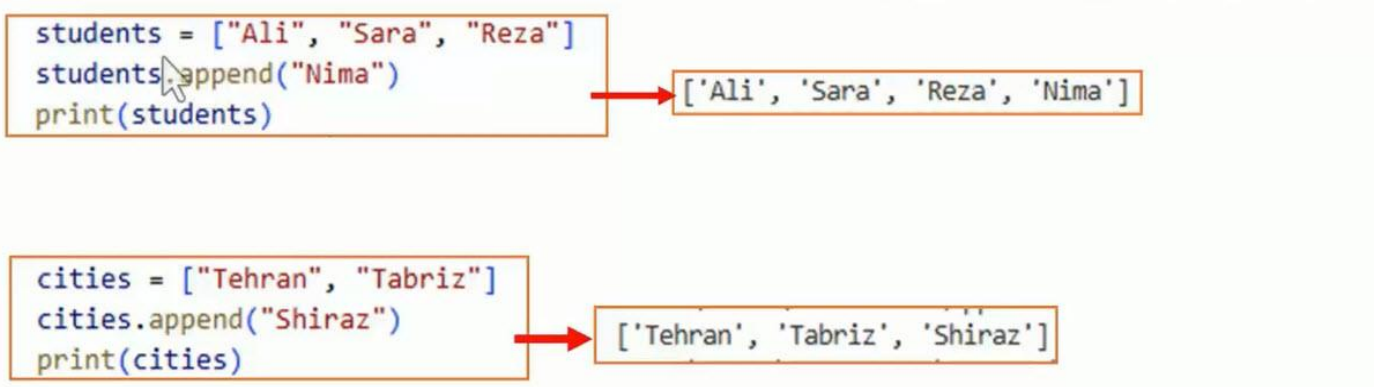
متد Append() یک عنصر را به انتهای لیست اضافه می کند،شکل زیر این موضوع را به خوبی نشان می دهد.



با توجه به شکل فوق متوجه می شویم که فرمت استفاده از متد Append() به صورت زیر می باشد.

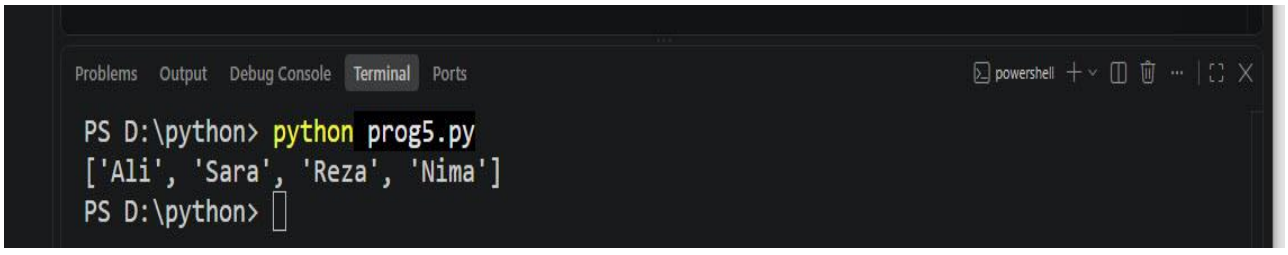
نام متغیر از نوع لیست . Append(آیتم جدید)

مثال:با استفاده از متد Append() شکل زیر را در برنامه پایتونی خود پیاده سازی نمایید.




```
students = ["Ali", "Sara", "Reza"]
students.append("Nima")
print(students)
```

خروجی:




```
cities = ["Tehran", "Tabriz"]
cities.append("Shiraz")
print(cities)
```

خروجی:

```
Problems  Output  Debug Console  Terminal  Ports

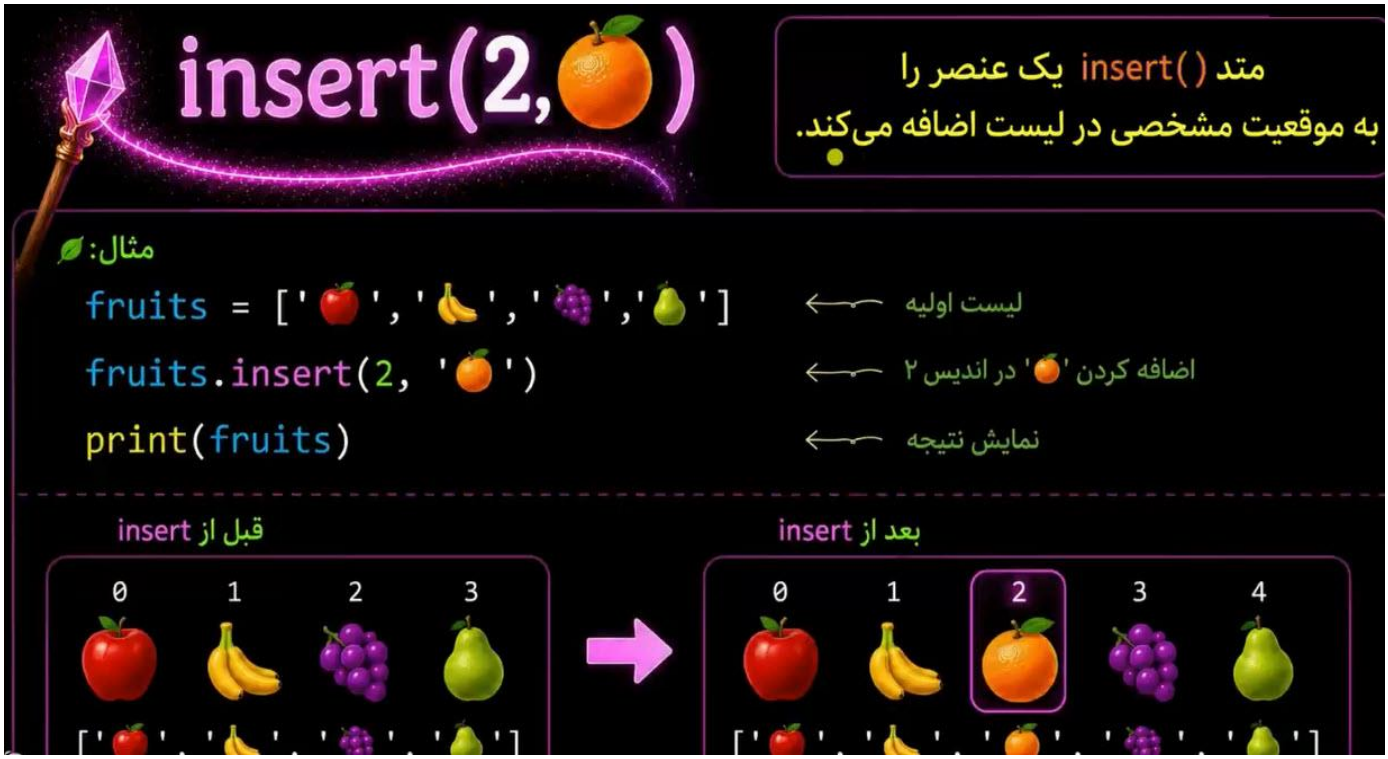
PS D:\python> python prog5.py
['Tehran', 'Tabriz', 'Shiraz']
PS D:\python>
```

نتیجه: تاکید میگردد که متد Append() یک عنصر را به انتهای لیست اضافه می کند.

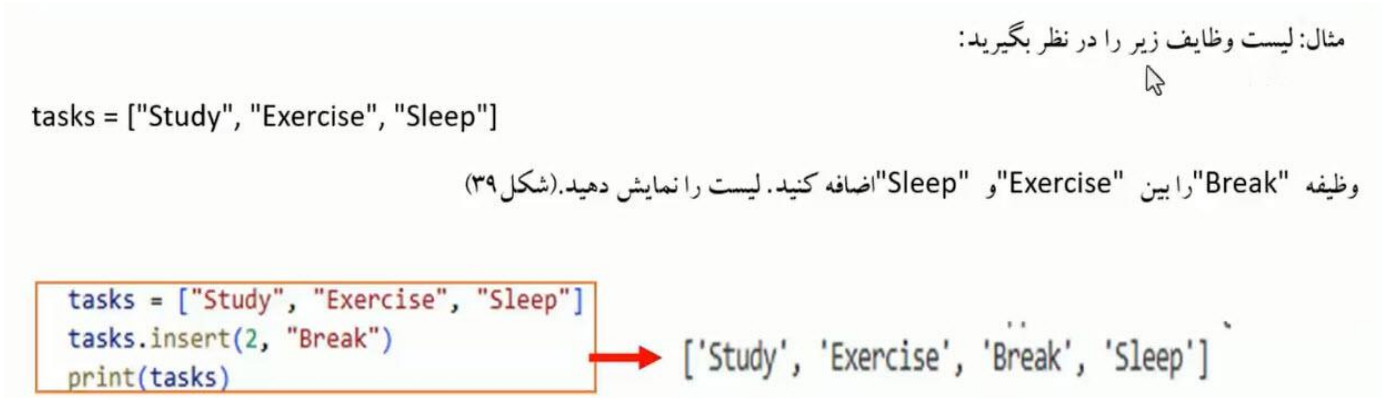
الف)می خواهیم مطابق شکل زیر آیتم جدیدی را به کمک متد insert در بین آیتم های لیست اضافه کنیم.



متد insert() یک عنصر را به موقعیت یا مکان مشخصی در لیست اضافه می کند(شکل زیر)،در واقع این متد آیتم ها را از مکانی که در دستور insert مشخص می کنیم به سمت راست حرکت یا Shift می دهد و آیتم جدید را در آن مکانی که مشخص کرده ایم برای ما اضافه می کند



مثال




```
tasks = ["Study", "Exercise", "Sleep"]
tasks.insert(2, "Break")
print(tasks)
```

خروجی

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\python> python prog5.py
['Study', 'Exercise', 'Break', 'Sleep']
PS D:\python>
```

نتیجه نهایی:



حذف عنصر از لیست

در پایتون برای حذف عناصر از یک لیست روشهای مختلفی وجود دارد. انتخاب روش مناسب به این بستگی دارد که بخواهیم بر اساس مقدار حذف کنیم یا بر اساس موقعیت یا اندازه. برای هر حالت یک مثال بیان می کنیم.

۱-حذف بر اساس مقدار — remove() اگر مقدار عنصر را بدانیم:

```
fruits = ["apple", "banana", "orange"]

fruits.remove("banana")

print(fruits)
```

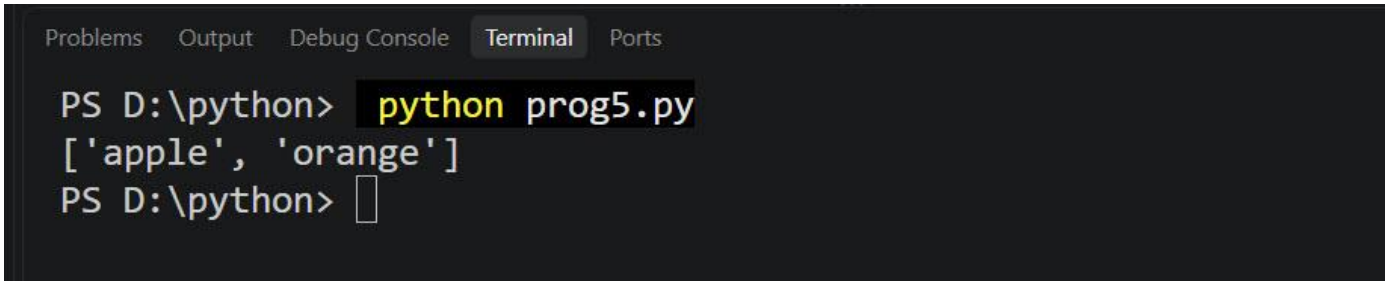


۲. حذف بر اساس موقعیت — pop() اگر موقعیت (اندیس) عنصر را بدانیم:

```
fruits = ["apple", "banana", "orange"]

fruits.pop(1)

print(fruits)
```



نکته: اگر در داخل پرانتز جلوی pop() چیزی نوشته نشود آخرین عنصر لیست حذف می‌شود.

مثال: یک لیست به نام Task داریم شامل عبارت Study, Exercise, Rest, Sleep. برنامه ای بنویسید که آخرین کار را از لیست حذف کند.

```
Task = ["Study", "Exercise", "Rest", "Sleep"]

Task.pop()

print(Task)
```

```
Problems  Output  Debug Console  Terminal  Ports
PS D:\python> python prog5.py
['Study', 'Exercise', 'Rest']
PS D:\python>
```

نکته: اگر داخل pop()عدد -۱ قرار بگیرد، آخرین عنصر لیست حذف می‌شود.

```
Task = ["Study", "Exercise", "Rest", "Sleep"]

Task.pop(-1)

print(Task)
```

```
Problems  Output  Debug Console  Terminal  Ports
PS D:\python> python prog5.py
['Study', 'Exercise', 'Rest']
PS D:\python>
```

نکته: متد clear() برای حذف تمام عناصر لیست استفاده می‌شود

```
Task = ["Study", "Exercise", "Rest", "Sleep"]

Task.clear()

print(Task)
```



نکته: دستور del برای حذف یک عنصر یا چند عنصر مشخص استفاده می‌شود

```
Task = ["Study", "Exercise", "Rest", "Sleep"]

del Task[1]

print(Task)
```

